Module 06 Lab - Regression and Classification Models

Objective: To understand the difference between regression and classification and to build your first linear models for both tasks.

In this lab, you will write the code to train and evaluate the models.

Part 1: Regression vs. Classification

This is the most fundamental distinction between types of supervised learning problems.

- **Regression:** The goal is to predict a **continuous numerical value**.
  - *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.
- **Classification:** The goal is to predict a **discrete category or class label**.
  - *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.

In this lab, we will tackle one of each.

Part 2: Linear Regression

Concept: Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".

- **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load the Titanic dataset
df = pd.read_csv(
    "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
)

# Remove rows with missing Age values
df.dropna(subset=["Age"], inplace=True)

# Select the features and target
features = ["Age", "Pclass"]
target_reg = "Fare"

X_reg = df[features]
y_reg = df[target_reg]

# Split the dataset into 80% training and 20% testing
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
```

Task 1: Train and Evaluate a Linear Regression Model

Your Task:

1. Create an instance of the `LinearRegression` model.
2. Train the model using the training data (`X_train_reg`, `y_train_reg`).
3. Make predictions on the test data.
4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
# 1. Create the Linear Regression model
lr_model = LinearRegression()

# 2. Train the model using training data
lr_model.fit(X_train_reg, y_train_reg)

# 3. Predict passenger fares using test data
y_pred_reg = lr_model.predict(X_test_reg)

# 4. Evaluate the model using Mean Squared Error
mse = mean_squared_error(y_test_reg, y_pred_reg)

# Display evaluation results
print(f"Mean Squared Error: {mse:.2f}")
print(f"Root Mean Squared Error: {np.sqrt(mse):.2f}")

# Display the learned coefficients
print("Model Coefficients:", lr_model.coef_)
print("Model Intercept:", lr_model.intercept_)
```

```
Mean Squared Error: 3364.92
Root Mean Squared Error: 58.01
Model Coefficients: [ -0.43695889 -38.29728935]
Model Intercept: 133.30726668128784
```

Part 3: Logistic Regression

Concept: Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.

- **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Make a copy of the dataset for classification
df_cls = df.copy()

# Convert Sex from text to numerical values
# Male = 0, Female = 1
df_cls["Sex"] = df_cls["Sex"].map({
    "male": 0,
    "female": 1
})

# Select features and target
features_cls = ["Age", "Pclass", "Sex"]
target_cls = "Survived"

X_cls = df_cls[features_cls]
y_cls = df_cls[target_cls]

# Split the data into training and testing sets
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls, y_cls, test_size=0.2, random_state=42
)
```

Task 2: Train and Evaluate a Logistic Regression Model

Your Task:

1. Create an instance of the `LogisticRegression` model.
2. Train the model using the classification training data.
3. Make predictions on the test data.
4. Evaluate the model using `accuracy_score`.

```
# 1. Create the Logistic Regression model
log_model = LogisticRegression(max_iter=1000)

# 2. Train the model using classification training data
log_model.fit(X_train_cls, y_train_cls)
```

```
# 3. Predict passenger survival using test data
y_pred_cls = log_model.predict(X_test_cls)

# 4. Evaluate the model using accuracy
accuracy = accuracy_score(y_test_cls, y_pred_cls)

# Display the result as a percentage
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

Knowledge Check

Instructions: Answer the following questions in this markdown cell.

1. **In your own words, what is the key difference between a regression problem and a classification problem?**

- The key difference between regression and classification is the type of output the model predicts. Regression predicts a continuous numerical value, such as the fare a Titanic passenger paid. Classification predicts a category or class label, such as whether a passenger survived or did not survive. In this lab, we use linear regression to predict passenger fares and logistic regression to classify passengers as survivors or non-survivors.

2. **The** `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?

- The numbers in lr_model.coef_ represent the coefficients for Age and Pclass. They show how much the predicted passenger fare changes when each feature increases by one unit while the other stays the same. A positive coefficient means the predicted fare increases, while a negative coefficient means it decreases.

3. **Why did we use** `mean_squared_error` **to evaluate the regression model but** `accuracy_score` **for the classification model?** Why wouldn't accuracy be a good metric for the fare prediction task?*

- We used mean squared error for linear regression because we were predicting numerical values, and MSE measures how far the predicted fares are from the actual fares. For logistic regression, we used accuracy because the model predicts categories, such as whether a passenger survived or not. Accuracy tells us how many predictions the model got correct. It would not work well for fare prediction because fares are continuous numbers, and even a small difference between the predicted and actual value would count as incorrect.